Práctico 6: APIs Rest

1. ¿Qué es una API y cuál es su principal objetivo dentro de un sistema de software?
2. ¿Qué significa REST ?
3. ¿Cuál es la diferencia entre una API y una API REST?
4. ¿Qué es un recurso en una API REST?
5. ¿Por qué REST se considera un estilo arquitectónico y no una tecnología?
6. ¿Qué significa que una API REST sea stateless?
7. ¿Cuáles son las principales ventajas de una arquitectura stateless?
8. ¿Cuál es la diferencia entre un recurso y su representación?
9. ¿Qué métodos HTTP son los más utilizados en una API REST y qué función cumple cada uno?
10. ¿Qué significa que una operación HTTP sea idempotente?
11. ¿Por qué POST generalmente no es idempotente?
12. ¿Qué son los códigos de estado HTTP y por qué son importantes?
13. ¿En qué situaciones deberían utilizarse los códigos 200, 201, 404 y 500?
14. ¿Qué características debería tener una URI bien diseñada en una API REST?
15. ¿Por qué se recomienda utilizar sustantivos y no verbos en las URIs REST?
16. ¿Qué es JSON y por qué se utiliza ampliamente en APIs REST?
17. ¿Qué es OpenAPI y cuál es su propósito principal?
18. ¿Qué ventajas ofrece el enfoque API-First o Contract-First?
19. ¿Qué información suele incluir una especificación OpenAPI?
20. ¿Qué es el modelo de información de una API?
21. ¿Por qué el modelo de información es importante antes de diseñar endpoints?
22. ¿Qué son las relaciones entre recursos? Proporcione un ejemplo.
23. ¿Cuál es la diferencia entre una colección de recursos y un recurso individual?

24. ¿Qué papel cumplen las APIs dentro de una arquitectura de microservicios?

25. ¿Qué problemas pueden surgir cuando existe un exceso de llamadas REST entre microservicios?

26. ¿Qué significa el principio de Data Ownership en microservicios?

27. ¿Por qué el diseño de APIs es considerado una decisión arquitectónica crítica en una arquitectura de microservicios?